Übungsblatt 4: Funktionen und Stack

Mikrocomputertechnik 1

Musterlösung

Aufgabe 1: Link und Ziel berechnen (10 Punkte)

RV32I ohne komprimierte Instruktionen. Jeder Fall startet unabhängig. Geben Sie Ziel-PC und ra danach an.

1. PC = `0x3000`: `jal ra, Ziel`, Ziel = `0x3700`. (2 P)
2. Gleicher PC und gleiches Ziel; alter ra = `0x4800`: `jal x0, Ziel`. (2 P)
3. PC = `0x3728`, ra = `0x3004`: `jalr x0, 0(ra)`. (2 P)
4. PC = `0x3440`, alter ra = `0x5009`: `jalr ra, -5(ra)`. (2 P)
5. `jalr x0, 1(t0)` mit t0 = `0x5803` beziehungsweise `0x5805`. Welches Ziel ist vierbyteausgerichtet? Gefragt ist die Zielrechnung, keine erfolgreiche Ausführung des ungültigen Ziels. (2 P)

Musterlösung

1. Ziel `0x3700`, ra = `0x3004`.
2. Ziel `0x3700`, ra bleibt `0x4800`.
3. Ziel `0x3004`, ra bleibt `0x3004`.
4. Alte Basis `0x5009` minus 5 ergibt `0x5004`; ra wird `0x3444`.
5. Ziele `0x5804` und `0x5806`; nur das erste ist durch vier teilbar. jalr löscht ausschließlich Bit 0.

Je Fall 2 P: Ziel und Link jeweils 1 P; im letzten Fall Ziele zusammen 1 P, Ausrichtung 1 P.

Aufgabe 2: Erhaltungspflichten begründen (8 Punkte)

Normale ILP32-Aufrufe. Entscheiden Sie, wer einen Wert erhalten muss und ob dafür hier eine zusätzliche Sicherung nötig ist. Je Fall 2 P.

1. Der Caller benötigt seinen t2-Zähler nach einem Helfer erneut.
2. Ein Leaf verändert s4, ruft aber niemanden auf.
3. Eine Funktion liest s3 als Konfigurationswert, verändert es aber nie.
4. Eine zurückkehrende Funktion ruft mit `jal ra, Helfer` weiter und benötigt anschließend ihre äußere Rückkehradresse.

Musterlösung

1. Der Caller muss den noch benötigten t2-Wert über den Aufruf erhalten; etwa im Stack, in einem geeignet geschützten Register oder durch Neuberechnung.
2. Der Leaf muss seinen s4-Eintrittswert wiederherstellen. Leaf-Eigenschaft befreit nicht von callee-saved-Pflichten.
3. Keine zusätzliche Sicherung fürs reine Lesen: Der Eintrittswert bleibt bereits erhalten.
4. Die Funktion ist gegenüber Helfer selbst Caller. Der innere jal überschreibt ra; der äußere Rückweg muss vor diesem Aufruf erhalten und später wiederhergestellt werden.

Je Fall Verantwortungszuordnung 1 P, begründeter Bedarf 1 P. Caller-saved bedeutet keine zugesicherte Erhaltung, nicht automatische Löschung.

Aufgabe 3: Ein größerer Frame (10 Punkte)

Gültiger Stack, Eintritt sp = `0xA000`. Eine Funktion muss ra und s0 bis s3, je 32 Bit, sichern. Es gilt die 16-Byte-Ausrichtung der ILP32-Konvention.

1. Bestimmen Sie Nutzbedarf, kleinste Framegröße, Padding und Reservierungsbefehl. (4 P)
2. Legen Sie ra, s0, s1, s2, s3 in dieser Reihenfolge bei Offsets 0, 4, 8, 12, 16 ab. Nennen Sie Slotadressen, Frame- und Paddingbereich. (3 P)
3. Ein innerer Aufruf reserviert weitere 16 Byte und nutzt Slot `4(sp)`. Berechnen Sie Adresse und Überlappung mit dem äußeren Frame. (2 P)
4. Dürfte eine andere Einwortfunktion Slot `8(sp)` ihres 16-Byte-Frames benutzen? (1 P)

Musterlösung

1. Nutzbedarf 20 Byte, Frame 32 Byte, Padding 12 Byte; `addi sp, sp, -32`. Je 1 P.
2. Neuer sp = `0x9FE0`; Slots `0x9FE0, 0x9FE4, 0x9FE8, 0x9FEC, 0x9FF0`. Frame `0x9FE0–0x9FFF`; Padding `0x9FF4–0x9FFF`. Slots 2 P, Bereiche zusammen 1 P.
3. Innerer sp = `0x9FD0`, Slot `0x9FD4`; keine Überlappung. Adresse 1 P, Begründung 1 P.
4. Ja, wenn der Slot innerhalb des reservierten Frames liegt, ausgerichtet ist und derselbe Slot wieder geladen wird. 1 P.

Aufgabe 4: Versatz mit Unteraufruf (14 Punkte)

`Versatz(a,b)=a-2b` ruft zuerst `Differenz(a,b)=a-b` auf und zieht danach das alte b ab. Vor `0x3400` gilt a0 = 19, a1 = 6, s1 = 77, sp = `0x9000`. Adressen sind feste Hex-Lehradressen; alle Befehle nativ.

```
3400  jal  ra, Versatz       Versatz = 3500
3404  addi t0, a0, 0
3500  addi sp, sp, -16
3504  sw   ra, 0(sp)
3508  sw   s1, 4(sp)
350C  addi s1, a1, 0
3510  jal  ra, Differenz     Differenz = 3600
3514  sub  a0, a0, s1
3518  lw   s1, 4(sp)
351C  lw   ra, 0(sp)
3520  addi sp, sp, 16
3524  jalr x0, 0(ra)
3600  sub  a0, a0, a1
3604  jalr x0, 0(ra)
```

1. Notieren Sie jeweils **danach** PC, a0, s1, ra und sp: `3400`, `3510`, `3604`, `3514`, `351C`, `3524`. PC bezeichnet die nächste Instruktion. (12 P)
2. Welche Wörter wurden bei `0x8FF0` und `0x8FF4` abgelegt? (2 P)

Musterlösung

nach	PC	a0	s1	ra	sp
3400	3500	19	77	3404	9000
3510	3600	19	6	3514	8FF0
3604	3514	13	6	3514	8FF0
3514	3518	7	6	3514	8FF0
351C	3520	7	77	3404	8FF0
3524	3404	7	77	3404	9000

1. Je Zelle 0,4 P, also 2 P pro Zeile. Adressspalten hexadezimal.
2. `0x8FF0: 0x00003404; 0x8FF4: 0x0000004D`. Je 1 P. Der innere Link überschreibt die Registerfassung von ra, nicht den äußeren Stackslot.

Aufgabe 5: Fehler reparieren (10 Punkte)

Jeder Fall ist unabhängig; nennen Sie erste falsche Wirkung und Korrektur. Je Fall 2 P.

1. Nach Reservierung von 32 Byte werden nur 16 Byte freigegeben; Eintritt sp = `0xA000`.
2. `sw s1, 4(sp)` wird durch `lw s1, 8(sp)` rückgängig gemacht; der Slot acht war unbekannt.
3. Im 16-Byte-Frame mit sp = `0x8FF0` wird erst freigegeben und danach `lw ra, 0(sp)` ausgeführt.
4. Versatz liefert 7 in a0, danach lädt der Epilog das alte erste Argument 19 wieder in a0.
5. Der folgende Ausschnitt sichert ra nicht. Beschreiben Sie den Ablauf ab der Rückkehr aus Differenz und reparieren Sie den Rückweg.

```
Fehler:
    jal ra, Differenz
Nach_Helfer:
    addi a0, a0, 1
    ret
```

Musterlösung

1. sp endet bei `0x9FF0` statt `0xA000`. Vollständige 32 Byte freigeben.
2. Der falsche Slot liefert nicht zugesichert den alten s1-Wert. Aus `4(sp)` laden.
3. Geladen wird von `0x9000` statt `0x8FF0`. Erst restaurieren, dann den Frame freigeben.
4. Das neue Ergebnis 7 wird durch 19 überschrieben. a0 als Rückgaberegister erhalten, statt das alte Argument blind zurückzuladen.
5. Differenz kehrt zu Nach_Helfer zurück. Dessen ret springt erneut dorthin, sodass die Addition wiederholt wird. Äußeren ra vor dem inneren jal sichern und vor dem abschließenden ret restaurieren.

Je Fall Diagnose und Reparatur je 1 P.

Aufgabe 6: Ein temporärer Wert überlebt (8 Punkte)

Im Hauptprogramm steht t2 = 8. `Spiegel` erhält a0 = −9, liefert 9 und verändert t2 ausdrücklich auf −20.

```
addi sp, sp, -16
sw   t2, 0(sp)
li   a0, -9
jal  ra, Spiegel
lw   t2, 0(sp)
addi sp, sp, 16
add  a0, a0, t2
```

1. Bestimmen Sie a0, t2 und sp am Ende bei Eintrittsstack S. (2 P)
2. Was berechnet die letzte Addition ohne die t2-Sicherung? (2 P)
3. Was entsteht, wenn zusätzlich das alte a0 = −9 vor der Addition restauriert wird? (2 P)
4. Eine zurückkehrende Funktion benutzt stattdessen s2 für die Acht. Welche Pflicht entsteht gegenüber ihrem eigenen Caller? (2 P)

Musterlösung

1. a0 = 17, t2 = 8, sp = S. Ergebnis 1 P, Erhaltung zusammen 1 P.
2. $9 + (-20) = -11$; der Caller darf den alten t2-Inhalt nicht voraussetzen. Rechnung und Begründung je 1 P.
3. $-9 + 8 = -1$ statt 17. Das neue Resultat wurde durch das alte Argument ersetzt. Rechnung und Diagnose je 1 P.
4. Sie muss den eigenen s2-Eintrittswert über ihre gesamte Aktivierung erhalten und vor der Rückkehr restaurieren. Pflicht und Umsetzungsidee je 1 P.

Aufgabe 7: Neue Funktionen in Venus

1. Implementieren Sie die Blattfunktion Spiegel mit a0 als Argument und Ergebnis: Vorzeichen durch Subtraktion von null wechseln. Testen Sie −9, 0 und 12 in einem vollständigen Hauptprogramm.
2. Implementieren Sie Versatz aus Aufgabe 4 mit Differenz als Helfer. Testen Sie $(19, 6)$, $(0, 0)$, $(-4, 3)$ und $(8, -5)$.
3. Setzen Sie s1 im Hauptprogramm auf 77. Notieren Sie am äußeren Rückkehrpunkt Ergebnis, s1 und sp; vergleichen Sie mit dem Eintrittszustand.
4. Prüfen Sie die tatsächliche Stackausrichtung der Umgebung und die nativen Rückkehradressen.
5. Venus-Ausgabe: Ergebnis nach a1 kopieren, a0 = 1, ecall; anschließend a0 = 10, ecall. Prüfen Sie vor dem Dienstaufruf.

Eine vollständige neue Referenz ist `beispiele/lab_04_versatz.s`.

Musterlösung

Spiegel: `sub a0, x0, a0`, danach `ret`; Ergebnisse 9, 0, −12. Der Leaf braucht keine Sicherung callee-saved Register.

Versatz-Ergebnisse: 7, 0, −10, 18. s1 wird jeweils wieder 77 und sp wieder S. Die Referenz verwendet Slots 0 und 4, bewahrt das zweite Argument und zieht es vom Helferergebnis ab. Die Rückkehr liegt vor der Ausgabevorbereitung. Werte sind Modellvorhersagen; die konkreten Venus-Anzeigen gehören in das eigene Protokoll.

Aufgabe 8: Rekursion mit konstantem Zuschlag (optional)

Definiert sei $F(0) = 2$ und $F(n) = F(n-1) + 3$ für $1 \leq n \leq 4$.

1. Schreiben Sie eine rekursive RV32I-Funktion mit n und Ergebnis in a0. Der Basisfall braucht keinen Frame; jeder andere Aufruf reserviert 16 Byte.
2. Welche Werte und maximale zusätzliche Stackreserve entstehen für n = 0, 2 und 4?
3. Warum muss n hier im Gegensatz zur rekursiven Summe nicht über den Unteraufruf hinweg erhalten bleiben?

Musterlösung

```
F:
    bne a0, x0, Rekursiv
    addi a0, x0, 2
    ret
Rekursiv:
    addi sp, sp, -16
    sw ra, 0(sp)
    addi a0, a0, -1
    jal ra, F
    addi a0, a0, 3
    lw ra, 0(sp)
    addi sp, sp, 16
    ret
```

Ergebnisse 2, 8, 14; maximale Zusatzreserve 0, 32, 64 Byte. Nach dem Unteraufruf wird nur die Konstante drei benötigt, nicht der alte n-Wert. Deshalb genügt hier die Sicherung von ra; s-Register werden nicht verändert.